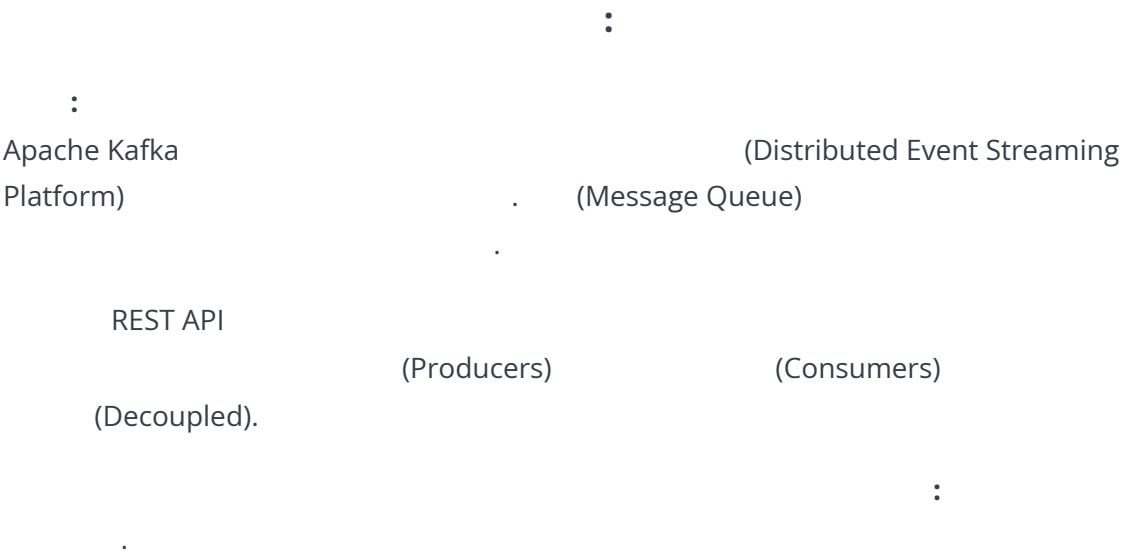


---

## . Apache Kafka

---



---

## : .Broker Topic Partition

### Consumer Group

|        |                           |
|--------|---------------------------|
|        |                           |
| Broker |                           |
| Topic  | stock-prices order-events |

|                |               |
|----------------|---------------|
| Partition      | (Parallelism) |
| Consumer Group | Consumer      |

Topic: stock-prices ( )

Consumer Group: "analytics-team"

Partition 0  
AAPL, TSLA

Partition 1  
MSFT, META

Partition 2  
GOOGL

Consumer 1

Consumer 2

Consumer 3

Consumer 2Consumer 13  
(Rebalance)

: Kafka RabbitMQ

:

|  | Kafka                | RabbitMQ        |
|--|----------------------|-----------------|
|  | Log-based ( )        | Queue-based ( ) |
|  | Consumer<br>(Replay) | ACK             |
|  | ( )                  |                 |
|  | Event Streaming      | Task Queue      |
|  |                      | Queue           |

## : Offset

:  
Offset (Sequential ID)  
. Offset bookmark — Consumer  
.  

- **Auto Commit:** offset ( )
- **Manual Commit:** offset commit  
( )  
Consumer offset  
—.

## : auto.offset.reset

earliest

latest

:  
Consumer offset :  

- **earliest :** —
- **latest :** : earliest latest  
.

## : Message Key

:  
Key .  
.  
: Key ( AAPL )  
.  
Key Round-Robin  
.

: Replication Factor

:

replication.factor=3

- **Leader:**
- **Followers:** (ISR — In-Sync Replicas)
- Leader Follower Leader

: replication.factor

: **acks=0** **acks=1** **acks=all** **Producer**

:

| acks=0   | Producer |  |                    |
|----------|----------|--|--------------------|
| acks=1   | Leader   |  | Leader Replication |
| acks=all | ISR      |  | —                  |

: acks=all

: **Consumer** **Producer** .

:

. : Consumer  
( Consumer )  
Consumer : Consumer Consumer Group  
: Consumer ( batch  
processing)

Consumer Lag: kafka-consumer-groups.sh Kafka UI

Consumer Lag = offset - Consumer offset

## : Exactly-Once Semantics

:  
:

- At-Most-Once:
- At-Least-Once:
- Exactly-Once:

Exactly-Once Idempotent Producer ( enable.idempotence=true )

Transactional API . Producer Transaction ID

## . Apache Spark (PySpark)

### : MapReduce (Hadoop)

:

Apache Spark . MapReduce:

|     | Hadoop MapReduce | Apache Spark            |
|-----|------------------|-------------------------|
|     |                  | (In-Memory)             |
|     | (I/O )           |                         |
| API | Java-centric     | Python/Scala/Java/R     |
|     | ( Batch)         | Structured Streaming    |
|     |                  | Spark Shell / Notebooks |

» : MapReduce

RAM / .»

---

## : RDD DataFrame

:

### RDD (Resilient Distributed Dataset):

- 
- 
- **Immutable:** RDD Lineage ( )
- **Resilient:** ( )
- API

### DataFrame:

- ( Pandas )
- Catalyst Optimizer query
- **DataFrame** RDD

```
# RDD -
rdd = sc.parallelize([1, 2, 3, 4])
result = rdd.filter(lambda x: x > 2).collect()

# DataFrame - ( )
df = spark.read.parquet("prices.parquet")
result = df.filter(df.price > 100).select("ticker", "price")
```

---

## : Lazy Evaluation

:

:

- **Transformation** ( ): filter() , select() , groupBy() — (Execution Plan)
- **Action** ( ): count() , show() , collect() , write() —

(Catalyst Optimizer).

```
# Transformation:
df2 = df.filter(df.price > 100) # Transformation
df3 = df2.select("ticker", "price") # Transformation
df4 = df3.groupBy("ticker") # Transformation

# Action:
df4.count() # !
```

## : SparkSession

```

:
SparkSession . Spark 2.0 SparkSession
SparkContext SQLContext .

```

```

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("FinancialAnalytics") \
    .master("local[*]") \ # CPU
    .getOrCreate()

# DataFrame SQL
df = spark.read.parquet("data/lake/")
df.createOrReplaceTempView("prices")
spark.sql("SELECT ticker, AVG(price) FROM prices GROUP BY ticker").show()

```

## : Spark SQL

```

:
Spark SQL query SQL DataFrame
. SQL ( Oracle/MySQL)
.

```

```

# DataFrame
df.createOrReplaceTempView("trades")

# SQL — Oracle!

```

```

result = spark.sql("""
SELECT
    ticker,
    DATE(timestamp) as trade_date,
    MIN(price) as low,
    MAX(price) as high,
    AVG(price) as avg_price,
    SUM(volume) as total_volume
FROM trades
WHERE ticker = 'AAPL'
GROUP BY ticker, DATE(timestamp)
ORDER BY trade_date DESC
""")

```

: Spark SQL      Catalyst Optimizer      —      query plan  
 (      Oracle Query Optimizer).

## : Structured Streaming

:  
 Structured Streaming  
 (Unbounded Table)      .

```

#      —      !
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "stock-prices") \
    .load()

#      —      Batch
prices = df.select(
    from_json(col("value").cast("string"), schema).alias("data")
).select("data.*")

#      Parquet —      micro-batch
query = prices.writeStream \
    .format("parquet") \

```



```

.option("path", "data/lake/prices") \
.option("checkpointLocation", "data/checkpoints") \
.trigger(processingTime="10 seconds") \
.start()

```

**Micro-Batch:**

(
  $N$ 
 ) Batch
 True Real-Time

## : Watermark Structured Streaming

:  
 Watermark (Late Data)

:  
 :

```

#
prices.withWatermark("timestamp", "5 minutes") \
  .groupBy(window("timestamp", "1 minute"), "ticker") \
  .agg(avg("price").alias("vwap"))

```

(State)

## : Partitioning

:

- : (Under-utilization)
- : (Overhead)
- - : CPU core

**Shuffle:**

groupBy join

```

#
df.rdd.getNumPartitions()

```

```
#
df.repartition(8) # Shuffle
df.coalesce(4) # Shuffle ( )
```

**Parquet** vs **CSV**

:

|       | CSV         | Parquet                        |
|-------|-------------|--------------------------------|
|       | (Row-based) | (Columnar)                     |
|       |             | (Snappy/GZIP) — x              |
|       |             | (Column Pruning)               |
|       | ( String)   | Schema (int, float, timestamp) |
| Query |             |                                |

price Parquet . CSV

```
# Parquet
df.write.partitionBy("date").parquet("data/lake/prices")

# — !
spark.read.parquet("data/lake/prices").filter("date = '2026-08-22'")
```

**cache()** vs **persist()**

:

- cache()** : DataFrame **RAM** ( persist(StorageLevel.MEMORY\_ONLY) )
- persist(level)** : — :
  - MEMORY\_ONLY** : RAM ( )

- MEMORY\_AND\_DISK : RAM
- DISK\_ONLY :

DataFrame :

```
# cache —
expensive_df = spark.read.parquet("huge_data").filter(...).groupBy(...)
expensive_df.count() #
expensive_df.show() # (! )

# cache —
expensive_df.cache()
expensive_df.count() # + RAM
expensive_df.show() # RAM ( !)
```

## : Catalyst Optimizer

:  
Catalyst query ( Oracle Query Optimizer).  
Spark SQL DataFrame API Catalyst:

. **Parsing:** query (Logical Plan)  
. **Analysis:**  
. **Optimization:** Predicate Pushdown Projection Pruning  
. **Physical Planning:** ( Broadcast Join  
vs Sort-Merge Join)

**Predicate Pushdown** : df.filter("date='2026-08-22']").select("price")  
Catalyst Parquet

## . Apache Airflow

: Airflow Cron Job

:  
Apache Airflow (Orchestration) .  
Cron:



```

extract = PythonOperator(task_id='extract', python_callable=extract_fn)
transform = PythonOperator(task_id='transform', python_callable=transform_fn)
load = PythonOperator(task_id='load', python_callable=load_fn)
validate = PythonOperator(task_id='validate', python_callable=validate_fn)

extract >> transform >> load >> validate

```

Extract      Transform      Load      Validate

## : Operator      Airflow

:  
Operator      (Task)      DAG      .      Operator      :

| Operator                |                  |
|-------------------------|------------------|
| PythonOperator          | Python           |
| BashOperator            |                  |
| SQLExecuteQueryOperator | query            |
| SparkSubmitOperator     | job      Spark   |
| EmailOperator           | (      )         |
| S3ToRedshiftOperator    | S3      Redshift |

**Sensor:**      Operator      :

- FileSensor :      Task      DAG
- ExternalTaskSensor :      Task      DAG
- HttpSensor :      API

## : XCom

:  
**XCom (Cross-Communication)**      Task      DAG      .

```
# Task 1:
def extract(**context):
    data = {'row_count': 15000, 'file_path': '/data/today.parquet'}
    context['ti'].xcom_push(key='extract_result', value=data)

# Task 2:
def transform(**context):
    result = context['ti'].xcom_pull(
        task_ids='extract',
        key='extract_result'
    )
    print(f"Processing {result['row_count']} rows")
```

[!WARNING]

**XCom** (metadata **XCom**).

: **schedule\_interval** **start\_date** **execution\_date**

:

:

- **start\_date** : DAG
- **schedule\_interval** : ( @daily , @hourly , cron  
2 0 \* \* \* )
- **execution\_date** : DAG )  
(!

:

start\_date=2026-08-01 schedule\_interval='@daily' :

- ( ) :
- execution\_date ( ) :

: **catchup**

:  
catchup=True start\_date Airflow  
DAG (Backfill).  
: start\_date=2026-01-01 — Airflow  
!  
• catchup=True :  
• catchup=False :( )

---

## : Airflow Executor

:  
Executor Task :

| Executor           |                    |        |
|--------------------|--------------------|--------|
| SequentialExecutor |                    |        |
| LocalExecutor      | Task               |        |
| CeleryExecutor     | Task Worker Celery | )<br>( |
| KubernetesExecutor | Task Pod           | )<br>( |

---

## : DAG

:  
. **Airflow UI:** DAG Task  
. **Task Instance Details:** / retry  
. **CLI:**

airflow tasks test dag\_id task\_id execution\_date

Task —  
. **XCom:** Task  
. **Connections:** /API

---

: Idempotency DAG

:  
Idempotent Task .

( Idempotent):

```
INSERT INTO fact_trades VALUES (...) -- = !
```

(Idempotent):

```
--  
DELETE FROM fact_trades WHERE trade_date = '{{ ds }}';  
INSERT INTO fact_trades SELECT ... WHERE trade_date = '{{ ds }}';
```

Airflow Task Retry . Idempotent

•

: OLTP OLAP

:

|       | OLTP                           | OLAP                         |
|-------|--------------------------------|------------------------------|
|       | Online Transaction Processing  | Online Analytical Processing |
|       | )<br>(                         |                              |
|       | Oracle MySQL PostgreSQL<br>( ) | Redshift BigQuery Snowflake  |
| Query | (INSERT/UPDATE<br>)            | (JOIN<br>)                   |
|       | 3NF ( )                        | (Star/Snowflake<br>Schema)   |



|      |              |      |
|------|--------------|------|
|      | ( )          | )    |
|      |              |      |
| » :  | Oracle MySQL | OLTP |
| OLAP | query        | .    |
|      |              | ,»   |

: Star Schema

:  
Star Schema (Dimensional Modeling) Fact  
Dimension :

|            |             |          |
|------------|-------------|----------|
| dim_stock  |             |          |
| stock_id   |             |          |
| ticker     |             |          |
| name       |             |          |
| sector     |             |          |
|            |             |          |
| dim_date   | fact_trades | dim_time |
|            |             |          |
| date_id    | trade_id    | time_id  |
| date       | stock_id    | hour     |
| quarter    | date_id     | minute   |
| day_of_wk  | time_id     | session  |
| is_weekend | price       |          |
|            | volume      |          |
|            |             |          |
| bid        |             |          |
| ask        |             |          |

- **Fact Table:** + ( )
- **Dimension Table:** ( )



: SCD (Slowly Changing Dimension)

:  
Dimension ( sector )

| SCD Type 1 | (Overwrite) | Sector                           |
|------------|-------------|----------------------------------|
| SCD Type 2 | /           | end_date +                       |
| SCD Type 3 |             | current_sector + previous_sector |

Type 2 :

```
-- dim_stock    SCD Type 2
| stock_id | ticker | sector | valid_from | valid_to | is_current |
|-----|-----|-----|-----|-----|-----|
| 1      | AMZN  | Retail | 2000-01-01 | 2024-06-30 | false    |
| 2      | AMZN  | Technology | 2024-07-01 | 9999-12-31 | true     |
```

: Columnar Storage OLAP

:  
Row Storage ( MySQL ) :

```
[AAPL, 187.5, 50M] [MSFT, 420.1, 30M] [GOOGL, 175.2, 25M]
```

Columnar Storage ( Parquet ) :

```
[AAPL, MSFT, GOOGL] [187.5, 420.1, 175.2] [50M, 30M, 25M]
```

OLAP:

- Column Pruning: ( )

- ( ) :
- **Vectorized Processing:** CPU SIMD

: AWS Redshift Google BigQuery Snowflake

:

|  | Redshift          | BigQuery             | Snowflake                   |
|--|-------------------|----------------------|-----------------------------|
|  | AWS               | Google Cloud         | Multi-cloud                 |
|  | Cluster-based ( ) | Serverless ( )       | Storage Compute             |
|  | ( )               | Query                | Storage + Compute           |
|  |                   |                      | ( Virtual Warehouses)       |
|  | Columnar ( )      | Columnar (Capacitor) | Columnar (Micro-partitions) |

**Snowflake:** Storage Compute  
.( ) .

: Data Lake Data Warehouse

:

|        | Data Lake                    | Data Warehouse             |
|--------|------------------------------|----------------------------|
|        | Raw ( ) —                    |                            |
| Schema | Schema-on-Read ( )           | Schema-on-Write ( )        |
|        | Data Scientists ML Engineers | Business Analysts BI Tools |
|        | Parquet JSON CSV             |                            |

|  |                     |                     |
|--|---------------------|---------------------|
|  | (Object Storage S3) | (Compute + Storage) |
|--|---------------------|---------------------|

**Data Lakehouse:** — Delta Lake Apache Iceberg. Schema ACID transactions Data Lake.

**: ETL ELT**

:

**ETL (Extract Transform Load):**

- 
- — ( Informatica SSIS)
- 

**ELT (Extract Load Transform):**

- 
- —
- BigQuery/Snowflake/Redshift

» : ELT Transform .»

**: Partitioning Clustering**

:

**Partitioning ( ):**

. ( )

query .

```
-- BigQuery:
CREATE TABLE fact_trades
PARTITION BY DATE(trade_timestamp)
AS SELECT * FROM raw_trades;

-- query :
SELECT * FROM fact_trades WHERE DATE(trade_timestamp) = '2026-08-22';
```

Clustering ( ): .

```
-- BigQuery: Cluster          ticker
CREATE TABLE fact_trades
PARTITION BY DATE(trade_timestamp)
CLUSTER BY ticker
AS SELECT * FROM raw_trades;

--          ticker          :
SELECT * FROM fact_trades
WHERE DATE(trade_timestamp) = '2026-08-22' AND ticker = 'AAPL';
```

: Data Quality    Data Validation

:  
:

. Schema Validation:

- . Null Check: null
- . Range Check: ( )
- . Uniqueness:
- . Freshness: ( > )
- . Volume Check:

: Great Expectations    dbt tests    Airflow data quality checks

```
#          Airflow
def validate_data(**context):
    row_count = context['ti'].xcom_pull(task_ids='load', key='row_count')
    if row_count < 100:
        raise ValueError(f"Too few rows: {row_count}. Expected >= 100.")
    if row_count > 1_000_000:
        raise ValueError(f"Suspiciously many rows: {row_count}.")
```

.

:

Yahoo Finance API

Kafka Producer      Kafka Topic (stock-prices)

Spark Streaming    Kafka Consumer  
(      VWAP)    (      )

Parquet Files (Data Lake)

Airflow DAG (      )  
    Extract:      Parquet  
    Transform: OHLCV  
    Load: PostgreSQL Star Schema  
    Validate:

FastAPI (Serving Layer)

Dashboard / BI Tool

:

- **Decoupling**
- (Streaming)      (Batch)
- Parquet
- Star Schema      query
- Airflow

**Consumer Lag (Lag)**

:

. : Consumer Lag Kafka UI Prometheus

. **Scale Out:** Consumer

. : (Batch Processing async I/O)

. **Backpressure:** Consumer catch up

.

---

**Parquet CSV**

:

- : GB CSV MB Parquet
- **Column Pruning:** price Parquet
- : Parquet (float64 timestamp) — CSV String
- **Predicate Pushdown:**
- / :

---

| <b>Batch Processing Stream Processing</b> |                     |                                     |
|-------------------------------------------|---------------------|-------------------------------------|
| :                                         |                     |                                     |
|                                           | Batch               | Stream                              |
|                                           | ( / )               |                                     |
|                                           |                     |                                     |
|                                           |                     | Micro-batch                         |
|                                           | OHLCV               |                                     |
|                                           | Spark Batch Airflow | Spark Streaming Kafka Streams Flink |

**Spark Structured Streaming = Micro-Batch:** —

.